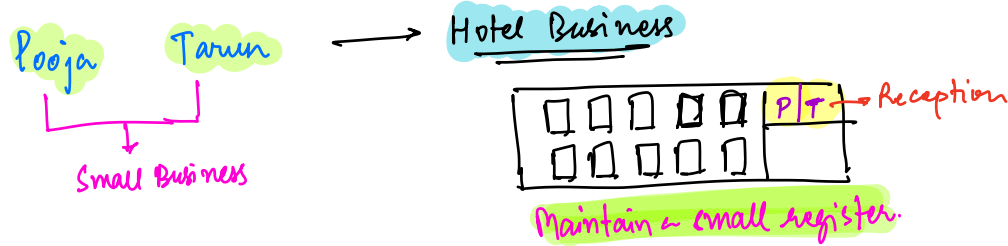


Hashing Introduction



Expansion to 1000 rooms

Room numbers :- $[1 \rightarrow 1000]$

→ bool check $[1001]$
 True (occupied) False (Vacant)

Room no. 1 → index 1
Room no. 1000 → index 1000

→ initially all the rooms are available.

→ check $[10] = \text{True}$ (Room no. 10 is now occupied)

2020-2021 comes → Pandemic

→ Numerology

→ 1000 lucky numbers for 1000 rooms.

 ↓
 $[1-10^9]$
 { 4, 7, 11, 19, 25, ..., 999, ... } → bool check $[10^9 + 1]$

Advantage :- Access time is $O(1)$

Disadvantage :- Huge space loss

→ Hashmap

HashMap <key, value>

↓
keys have to be unique in a Hashmap

e.g. HM {
 <79, occupied>
 <100, occupied>
 <124, occupied>
}

→ If you want to store only key and no values, then you can use a HashSet <key>

Q1 store population of every country

HashMap <key, value>

↓ ↓
country population
String long

HashMap <String, long>

Q2 No. of states in each country

HashMap <key, value>

↓ ↓
country name No. of states
String int

HashMap <String, int>

Q3 for every country, we want to store all states names

HashMap <key, value>

↓ ↓
country name state names
String List/Array

→ HashMap <String, List<String>>

Q4 For every country, store population of each state.

HashMap <key, value>

↓
country
name
String

↘ population of each state

HashMap <state Name, population>

HashMap <String, long>

HashMap <String, HashMap <String, long>>

↓
country
name

↓
state
name

↓
population

India → { UP → 1000
Karnataka → 2000
MP → 1000

USA → { Virginia → 100
Michigan → 100
NY → 100

In a hashmap,

→ Value → can be anything

→ Key → {String, int, long, float}

Can we have list/array as a key? → No

Inbuilt hashmap functionalities hm <Key, value>

1) insert <Key, value>

2) search <Key>

3) remove <Key, value>

4) update <key, new value>

5) size() → No. of keys in your hashmap

T.C = $O(1)$ → Average Case T.C.

S.C = $O(N)$

→ Hashset <Keys>

① insert <Key>

② search <key>

③ remove <key>

④ modify (key) →
 → remove (old key)
 → insert (new key)

⑤ size () → no. of keys/elements in your hashset

for a single operation, T.C = $O(1)$ → avg. case
 S.C = $O(N)$

HashMap & HashSet

	Java	C++	JavaScript	Python	C#
HashMap	HashMap	unordered_map	map	Dict	dict
HashSet	HashSet	unordered_set	set	set	hashset

for discussion, I will
 → HashSet → <K>
 → HashMap → <K, V>

Q1) Find frequency of Numbers

Given N array elements and Q queries.

For each query, find frequency of an element in array.

$N=10$, $arr[10] = \{2, 6, 3, 8, 2, 8, 2, 3, 8\}$

Query: 4

2:- 3

8:- 3

3:- 2

5:- 0

Brute force sol:-

→ For every query, iterate on $arr[i]$ and get count.

T.C = $O(QN)$

S.C = $O(1)$

HashMap <key, value>

array elements
(int)

frequency
(int)

→ HashMap <int, int> hm;

$arr[10] = \{2, 6, 3, 8, 2, 8, 2, 3, 8\}$

hm:-

```
{2, 1} → {2, 2} → {2, 3}
{6, 1}
{3, 1} → {3, 2}
{8, 1} → {8, 2} → {8, 3}
```

Pseudo code:-

HashMap <int, int> hm;

for (int i=0; i<N; i++) {

if (arr[i] is in hm) {

// update value by 1

hm[arr[i]]++;

}

else {

// insert {arr[i], 1} in hm

hm.insert(arr[i], 1);

}

}

T.C = $O(N)$

S.C = $O(N)$

Query: 4

2:- $hm[2] = 3$

8:- $hm[8] = 2$

3:- $hm[3] = 2$

5:- $hm[5] = \rightarrow$ Exception (first search whether present or not, then only get its value)

```
for (int i=1; i <= q; i++) {
```

```
    // read x
```

```
    if (x is in hm) {
```

```
        // alters value of x
```

```
        print(hm[x]);
```

```
    }
```

```
    else {
```

```
        print(0);
```

```
    }
```

```
}
```

Total T.C = $O(Q+N)$

S.C = $O(N)$

Qⁿ Find the first non-repeating element

e.g. $arr[6] = \{1, 2, 3, 1, 2, 5\} \rightarrow Ans = 3$

$arr[8] = \{4, 3, 3, 2, 5, 6, 4, 5\} \rightarrow Ans = 2$

$arr[7] = \{2, 6, 8, 4, 7, 2, 9\}$

Idea:-

- 1) Insert all elements in hm
- 2) Iterate on hm, get the first key with freq. > 1

Idea:-

- 1) Insert all elements in hm
- 2) Iterate on i/p $arr[]$ and get element with freq. $= 1$

Note:- In hm, insertion order is not maintained

$$T.C = O(N + N) = O(N)$$

$$S.C = O(N)$$

TODO:- Code this.

Q3 Given N array elements, find the no. of distinct elements.

e.g. $arr[5] = \{3, 5, 6, 5, 7\}$ $ans = 4$

↓ ↓ ↓ ↓ ↓
+1 +1 +1 X +1

Quiz $arr[7] = \{6, 3, 7, 3, 8, 6, 9\}$ $Ans = 5$

↓ ↓ ↓ ↓ ↓ ↓ ↓
+1 +1 +1 X +1 X +1

Hashset $\langle \overset{\text{int}}{\text{key}} \rangle \text{hs}$
↳ array elements

6

3

7

3 → Will you insert this? → Even if you insert it, it does not matter.

8

6

9

// Pseudo code

Hashset $\langle \text{Integer} \rangle \text{hs};$

for (int i=0; i < n; i++) {

$\text{hs.insert}(arr[i]);$

}

return $\text{hs.size}();$

T.C = $O(N)$

S.C = $O(N)$

Q4 Given an array elements. Check if all elements are distinct or not.

Idea:- 1) Insert all elements in a hashset, $h1$.

```
2) if (h1.size() == N){  
    | //all elements are distinct  
    }  
    else{  
    | //There is repetition.  
    }
```

Q5 Given N array elements, check if there exists a subarray with $\text{sum} = 0$.

arr[] =

0	1	2	3	4	5	6	7	8	9
2	2	1	-3	4	3	1	-2	-3	2

Brute force:-

1) For every subarray, you generate sum and check if $\text{sum} = 0$.

$$T.C = O(N^3)$$

2) Finding all subarray sums using carry forward $\rightarrow T.C = O(N^2)$
 $S.C = O(1)$

arr[] =	0	1	2	3	4	5	6	7	8	9
	2	2	1	-3	4	3	1	-2	-3	2
pf[] =	2	4	5	2	6	9	10	8	5	7

$$pf[2] = 5$$

$$pf[8] = 5$$

$$\rightarrow pf[8] = \text{sum}[0-2] + \text{sum}[3-8]$$

$$5 = 5 + \text{sum}[3-8]$$

$$\text{sum}[3-8] = 0$$

$$pf[0] = 2$$

$$pf[3] = 2$$

$$[0-3] = [0-0] + [1-3]$$

$$2 = 2 + \text{sum}[1-3]$$

$$\text{sum}[1-3] = 0$$

If there is a repetition in pf[], there exists a subarray with $\text{sum} = 0$.

1) Construct $pf[] \longrightarrow O(N)$

2) `HashSet<int> hs`

insert all $pf[]$ elements in $hs. \longrightarrow O(N)$

```
3) if (hs.size() < N || 0 is in hs) {  
    // repetition  
    return true  
}  
else {  
    return false  
}
```

✓

$$T.C = O(2N) = O(N)$$

$$S.C = O(2N) = O(N)$$

e.g. $arr[3] = 3 \quad 0 \quad 4$
 $pf[] = 3 \quad 3 \quad 7 \longrightarrow$ Works for this

e.g. $arr[4] = -1 \quad 4 \quad -3 \quad 2$
 $pf[] = -1 \quad 3 \quad 0 \quad 2 \longrightarrow$ No repetition
 $\longrightarrow$ If $pf[]$ contains a 0.